

Lab 10: RISC-V Vector ISA Lab

In this lab, you will use the NEMU simulator and the Nexus-AM bare-metal runtime to study the RISC-V Vector Extension (RVV). You will inspect how vector instructions are modeled in NEMU, complete RVV intrinsic code, and run the program on NEMU.

By the end of this lab, you will be able to:

1. Configure NEMU with RVV support enabled
2. Identify where NEMU implements key RVV concepts such as vector length, masks, and memory access patterns
3. Use RVV C intrinsics to implement vectorized matrix-vector multiplication
4. Use RVV masks to implement branchless string case conversion
5. Compare scalar and vector implementations by running a Nexus-AM program on NEMU

Default environment configuration:

- **NEMU:** OpenXiangShan NEMU at commit [0860de24253ef2097663fe64095b0fc329515489](#)
- **Nexus-AM:** Bare-metal runtime used to build and run the test program
- **Target:** `ARCH=riscv64-xs`
- **Required NEMU option:** `CONFIG_RVV=y`

Note: After setting `NEMU_HOME` or `AM_HOME`, use the same terminal for the remaining commands or add the variables to your shell profile.

Step 1 - Environment Setup

Background NEMU executes the RISC-V binary, while Nexus-AM provides the bare-metal application framework. Before writing RVV code, verify that NEMU is built with RVV enabled and that Nexus-AM can run a simple program.

Install and build NEMU:

```
git clone git@github.com:OpenXiangShan/NEMU.git
cd NEMU
git checkout 0860de24253ef2097663fe64095b0fc329515489
git submodule update --init --recursive
export NEMU_HOME=${PWD}

sudo apt install build-essential man gcc gdb git libreadline-dev libsdl2-
dev zstd libzstd-dev bison flex
make riscv64-xs_defconfig
make
```

Check that RVV is enabled:

```
grep "RVV" -ri .config
```

Expected output:

```
CONFIG_RVV=y  
CONFIG_RVV_AGNOSTIC=y
```

Install and test Nexus-AM:

```
git clone https://github.com/OpenXiangShan/nexus-am.git --depth=1  
cd nexus-am  
export AM_HOME=${PWD}  
  
cd apps/coremark  
make ARCH=riscv64-xs  
${NEMU_HOME}/build/riscv64-nemu-interpreter -b ./build/coremark-riscv64-  
xs.bin
```

Step 2 - Explore RVV in NEMU

Background RVV behavior is controlled by architectural state such as vector registers, vector length (`vl`), element width, masks, and memory access modes. Explore the relevant NEMU source code and verify your answers against the implementation.

AI tools are optional. If used, report the tool, model, prompts, and a brief usefulness/reliability reflection.

Question 1: Identify the source files or functions that implement vector registers and `vl`. Explain how `vl` affects the number of elements executed by one RVV instruction.

Question 2: Identify the source files or functions that implement vector masks. Explain how a masked operation differs from an unmasked operation in NEMU.

Question 3: Identify where NEMU handles strided or indexed vector memory access. Explain why these access patterns are useful for data-level parallelism.

Source Exploration Record: After Questions 1-3, report how much time you spent on this source-code exploration. If you used AI tools, also attach:

1. Tool name and model used
 2. Prompts or questions submitted
 3. What the AI tool helped you find
 4. What you verified manually in the NEMU source code
 5. Any mistakes, limitations, or useful lessons from using the tool
-

Step 3 - Write and Run an RVV Program

Background RVV C intrinsics look like normal C functions, but the compiler maps them to RVV instructions. In this step, you will complete two vectorized functions and compare their results with scalar implementations:

1. Matrix-vector multiplication: $y = A * x$
2. Branchless lowercase-to-uppercase string conversion using masks

Create or replace `hello.c` in your Nexus-AM application directory with the following skeleton. Fill in every `TODO` section.

```
#include <klib.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

#if defined(__riscv_vector)
#include <riscv_vector.h>
#endif

//
=====
=
// Task A: Matrix-Vector Multiplication  $y = A * x$ 
//
=====
=

#define MATRIX_SIZE 4

static int matrix_a[MATRIX_SIZE][MATRIX_SIZE] = {
    {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, 16}
};
static int vector_x[MATRIX_SIZE] = {1, 2, 3, 4};
static int result_scalar[MATRIX_SIZE] = {0};
static int result_vector[MATRIX_SIZE] = {0};

void matrix_vector_scalar(int y[MATRIX_SIZE], const int A[MATRIX_SIZE]
[MATRIX_SIZE], const int x[MATRIX_SIZE]) {
    for (int i = 0; i < MATRIX_SIZE; i++) {
        y[i] = 0;
        for (int j = 0; j < MATRIX_SIZE; j++) {
            y[i] += A[i][j] * x[j];
        }
    }
}

#if defined(__riscv_vector)
void matrix_vector_vector(int y[MATRIX_SIZE], const int A[MATRIX_SIZE]
[MATRIX_SIZE], const int x[MATRIX_SIZE]) {
    for (int i = 0; i < MATRIX_SIZE; i++) {
```

```

// TODO 1: Set vl for 32-bit elements with LMUL=1.
size_t vl = /* YOUR CODE HERE */;

// TODO 2: Load row A[i] and vector x.
vint32m1_t vec_row = /* YOUR CODE HERE */;
vint32m1_t vec_x    = /* YOUR CODE HERE */;

// TODO 3: Multiply vec_row and vec_x.
vint32m1_t vec_mul = /* YOUR CODE HERE */;

int32_t sum = __riscv_vmv_x_s_i32m1_i32(
    __riscv_vredsum_vs_i32m1(vec_mul, __riscv_vmv_s_x_i32m1(0, vl),
vl)
    );

    y[i] = sum;
}
}
#else
void matrix_vector_vector(int y[MATRIX_SIZE], const int A[MATRIX_SIZE]
[MATRIX_SIZE], const int x[MATRIX_SIZE]) {
    matrix_vector_scalar(y, A, x);
}
#endif

//
=====
=
// Task B: Branchless String Case Conversion
//
=====
=

#define TEST_STR_LEN 256

static char test_string_scalar[TEST_STR_LEN];
static char test_string_vector[TEST_STR_LEN];

void init_test_strings() {
    strcpy(test_string_scalar, "Hello, World! This is a Test String for
RISC-V Vector Extension. 123456789");
    strcpy(test_string_vector, "Hello, World! This is a Test String for
RISC-V Vector Extension. 123456789");
}

void string_to_upper_scalar(char* str, int len) {
    for (int i = 0; i < len; i++) {
        if (str[i] >= 'a' && str[i] <= 'z') {
            str[i] = str[i] - 32;
        }
    }
}

}

#endif defined(__riscv_vector)

```

```

void string_to_upper_vector(char* str, int len) {
    size_t vl;

    for (; len > 0; len -= vl, str += vl) {
        // TODO 4: Set vl for 8-bit elements.
        vl = /* YOUR CODE HERE */;

        // TODO 5: Load characters from str.
        vuint8m1_t vec_chars = /* YOUR CODE HERE */;

        vuint8m1_t vec_a = __riscv_vmv_v_x_u8m1('a', vl);
        vuint8m1_t vec_z = __riscv_vmv_v_x_u8m1('z', vl);

        // TODO 6: Create mask for 'a' <= c <= 'z'.
        vbool8_t mask_ge = /* YOUR CODE HERE */;
        vbool8_t mask_le = /* YOUR CODE HERE */;
        vbool8_t mask = __riscv_vmand_mm_b8(mask_ge, mask_le, vl);

        vuint8m1_t vec_delta = __riscv_vmv_v_x_u8m1(32, vl);

        // TODO 7: Subtract 32 only for masked lanes.
        vec_chars = /* YOUR CODE HERE */;

        // TODO 8: Store characters back to str.
        /* YOUR CODE HERE */;
    }
}

#else
void string_to_upper_vector(char* str, int len) {
    string_to_upper_scalar(str, len);
}
#endif

int verify_results(const int result1[MATRIX_SIZE], const int
result2[MATRIX_SIZE]) {
    for (int i = 0; i < MATRIX_SIZE; i++) {
        if (result1[i] != result2[i]) return 0;
    }
    return 1;
}

int main() {
    printf("=== RISC-V Vector Extension Lab ===\n\n");

    printf("Task A: Matrix-Vector Multiplication\n");
    matrix_vector_scalar(result_scalar, matrix_a, vector_x);
    matrix_vector_vector(result_vector, matrix_a, vector_x);

    if (verify_results(result_scalar, result_vector)) {
        printf("Task A PASSED: Scalar and vector implementations
match!\n");
    } else {
        printf("Task A FAILED: Results do not match!\n");
    }
}

```

```

    printf("\nTask B: String Case Conversion\n");
    init_test_strings();
    string_to_upper_scalar(test_string_scalar, strlen(test_string_scalar));
    string_to_upper_vector(test_string_vector, strlen(test_string_vector));

    if (strcmp(test_string_scalar, test_string_vector) == 0) {
        printf("Task B PASSED: Scalar and vector implementations
match!\n");
    } else {
        printf("Task B FAILED: Results do not match!\n");
    }

    return 0;
}

```

Build and run the program:

```

cd $AM_HOME/apps/hello
make ARCH=riscv64-xs
${NEMU_HOME}/build/riscv64-nemu-interpretor -b ./build/hello-riscv64-xs.bin

```

Expected output includes:

```

Task A PASSED: Scalar and vector implementations match!
Task B PASSED: Scalar and vector implementations match!

```

Question 4: Submit your completed `hello.c`. Briefly explain which RVV intrinsics you used for load, multiply, reduction, comparison, masked subtraction, and store.

Question 5: Submit a screenshot showing the successful NEMU execution output. Immediately after the program finishes, run `whoami` in the same terminal and include the output in the screenshot.

Question 6: Compare the scalar and vector string conversion implementations. Explain why the RVV version can avoid branches while still updating only lowercase letters.

References

- [RISC-V Vector Extension Specification](#)
- [RISC-V Vector Intrinsics](#)
- [OpenXiangShan NEMU](#)
- [OpenXiangShan Nexus-AM](#)